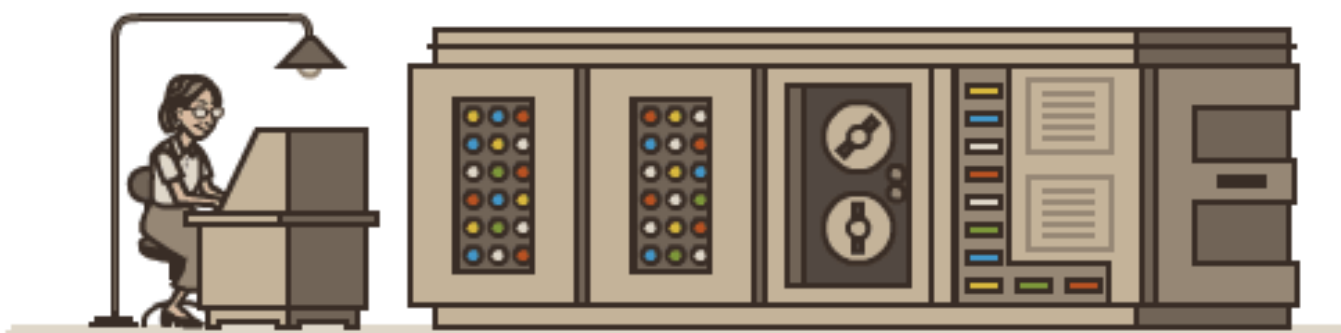

19 软件测试

Outline

- Bug Story
- 什么是软件测试？
- 软件测试的分类
- 软件测试技术
- 软件测试活动
- 软件测试的度量



First Bug

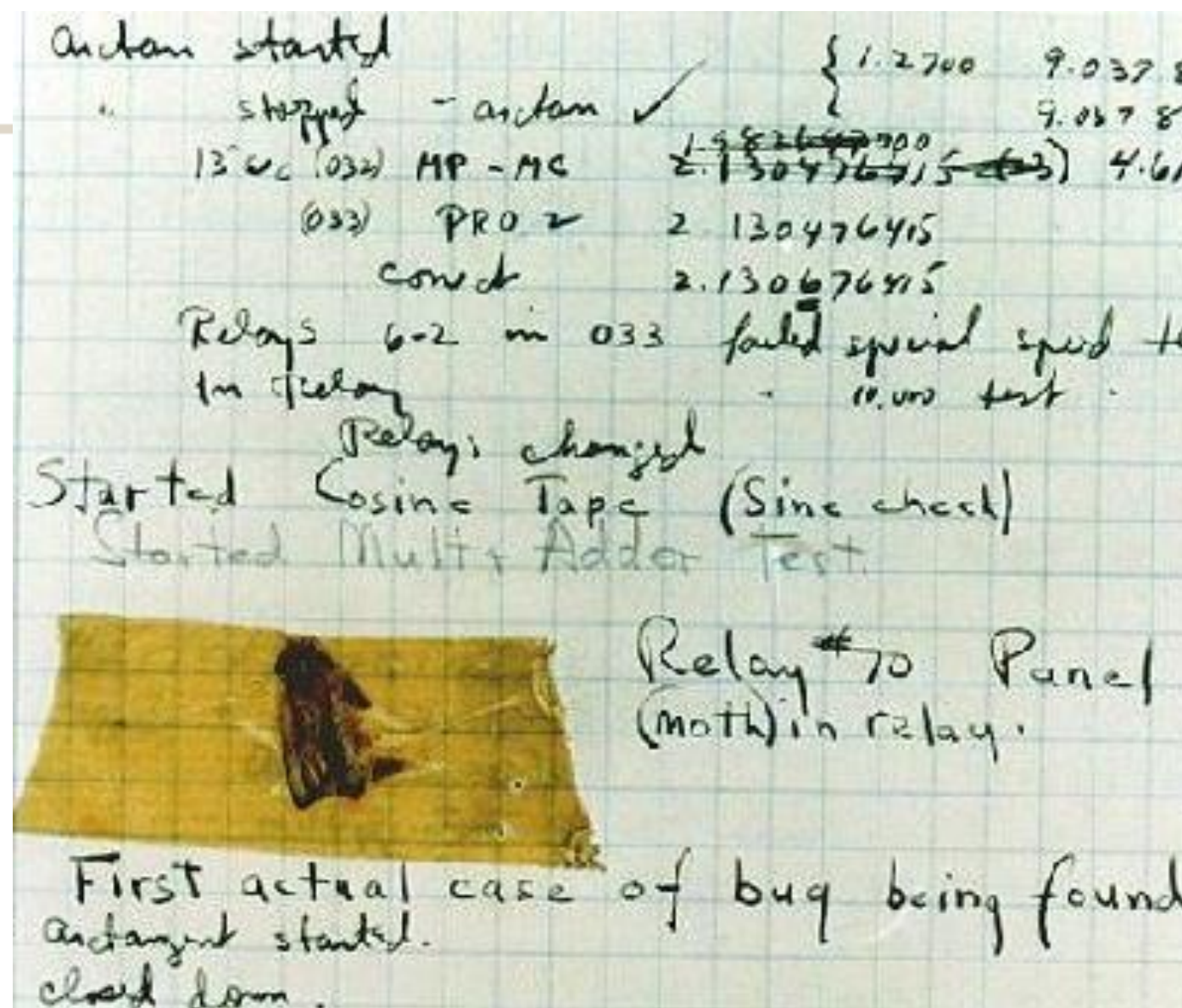
Grace Murray Hopper(1906-1992)

计算机科学家 美国海军将军

编译器的发明者

COBOL语言的开发负责人

ACM Grace Murray Hopper Award



Millennium bug (Y2K)

- In the 1970s and on, programmers saved precious memory and disk space by storing years in a two-digit format
 - They (we) never dreamed that their programs would still be running in 2000



Radiation machine kills four: 1985 to 1987

- Faulty software in a Therac-25 radiation-treatment machine resulted in several cancer patients receiving lethal overdoses of radiation. Four patients died
- An investigation by independent scientists found that accidents occurred even after Atomic Energy of Canada Limited (AECL) thought it had fixed particular bugs
- They wrote in their report, "The basic mistakes here involved poor software-engineering practices and building a machine that relies on the software for safe operation."



AT&T long distance service fails: 1990

- Switching errors in AT&T's call-handling computers caused the company's long-distance network to go down for nine hours, the worst of several telephone outages in the history of the system
- The meltdown affected thousands of services and was eventually traced to a single faulty line of code



at&t

Patriot missile defense 1991

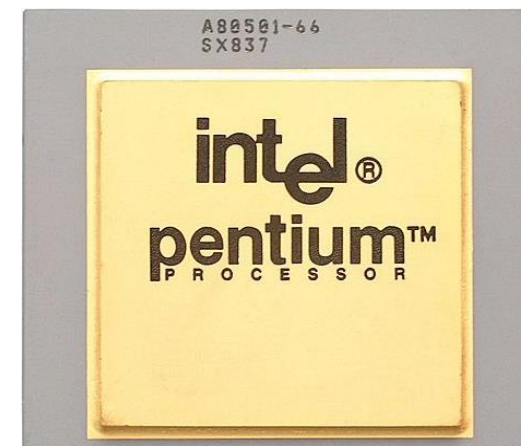
- The Patriot missile defense system failed to defend against several missiles
- A small timing error in the system's clock accumulated so that after 14 hours, the tracking system was no longer accurate
 - In the attack on Dhahran, KSA, the system had been in operation for over 100 hours
- The system was mobile, so the developers did not expect it to remain in one place for 14 hours



Pentium FDIV bug

$$(4195835/3145727) * 3145727 - 4195835 \neq 0$$

- Test engineers found bug
 - Management decided it was not severe
- A user reported it 10/30/1994, Intel attempted to play down the bug. Offered to replace chips if you could prove you were affected
- Ultimately, they apologized; replaced all chips, cost \$450 million
- Interestingly, 1.13 MHz Pentium III caused software freezes



Disney's "Lion King Animated Storybook" 1994

- Disney released its first multimedia CD-ROM for children
- A major ad campaign touting it as “the game to buy”
- After Christmas Day, many angry parents were on the phone to Disney with crying children in the background
- The problem was the game worked on only a few platforms and nothing was said about this on the wrapper as Disney hadn't realized the problem



什么是对的软件？

- Are you building it “right”?
- Are you building the “right” thing?

Are you building it “right”?

- Software Correctness
 - A program P is considered with respect to a specification S , if and only if:
 - For each valid input, the output of P is in accordance with the specification S
- Verification

Are you building the “right” thing?

- 常我们会将软件与终端用户或者其他利益相关者的需求进行比较。
- Validation

Verification and Validation

- Verification: is the system correct with respect to some specification?
- Validation: did we build the right system?
- V&V differences don't matter
- V&V generally refers to any activity that attempts to ensure that the software will function as required

If software not correct...

- Definitions
 - Fault / Defect – The cause of a failure, e.g., a missing or incorrect piece of code
 - Error – An intermediate unstable state
 - Failure – The manifested inability of the program to perform the function required, i.e., a system malfunction evidenced by incorrect output, abnormal termination or unmet time and space constraints.
- Relationship
 - A fault may remain undetected long time, until some event activates it.
 - If an when the error propagates to the output, it eventually causes the failure.
- Fault Error Failure
 - This chain can recursively iterate: a fault in turn can be caused by the failure of some other interacting system.

Common V&V Activities

- Static Analysis Techniques –
 - Based solely on the (manual or automated) examination of project documentation of software models and code, and of other related information about requirements and design
 - Generally yields valid results, but may be weak in precision
- Dynamic Test Techniques –
 - Exercise the software in order to expose possible failures
 - Behavioral and performance properties of the program are also observed
 - Yields more precise results, but only holding for the examined executions

大模型辅助静态代码分析

大模型作为智能静态分析工具

- 传统静态分析：基于规则的语法/语义检查；大模型：基于语义理解的深度分析
- 大模型能理解代码意图，发现传统工具难以检测的逻辑缺陷和设计问题

大模型静态分析的典型能力

- 代码审查：识别命名不规范、逻辑错误、潜在空指针、资源泄漏
- 安全扫描：检测 SQL注入、XSS、缓冲区溢出等安全漏洞
- 规范一致性：检查代码是否符合接口契约、前置/后置条件
- 文档一致性：对比代码实现与注释/规格说明是否一致

工具推荐

- GitHub Copilot、CodeRabbit、SonarQube + LLM、Snyk Code（AI安全扫描）

Prompt 示例

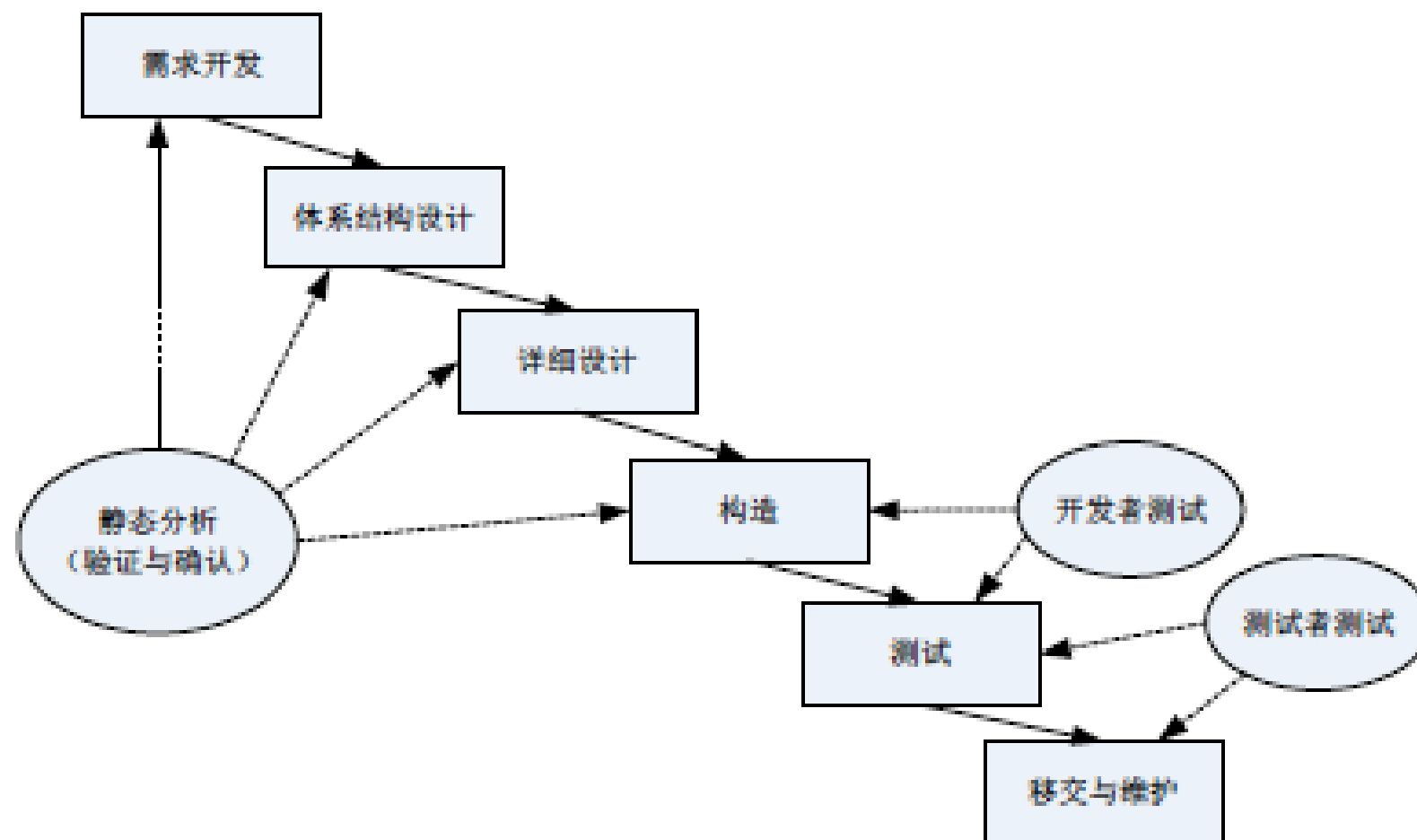
- "请对以下Java代码进行静态分析，识别潜在的缺陷、安全漏洞和不符合规范的写法，并给出修复建议。"

Static Techniques

- Can be applied at the requirements phase; at the design phase; and during the implementation phase
- Traditional Static Techniques – heavily manual, error-prone, time consuming
 - Software inspection – The step-by-step analysis of the documents (deliverables) produced, against a compiled checklist of common and historical defects
 - Software reviews – The process by which different aspects of the work product are presented to project personnel (managers, users, customer etc) and other interested stakeholders for comment or approval
 - Code reading – The desktop analysis of the produced code for discovering typing errors that do not violate style or syntax
 - Algorithm analysis and tracing – The process in which the complexity of algorithms employed and the worst case, average-case and probabilistic analysis evaluations can be derived
- Static Analysis Techniques relying on use of Formal Methods

Dynamic Techniques

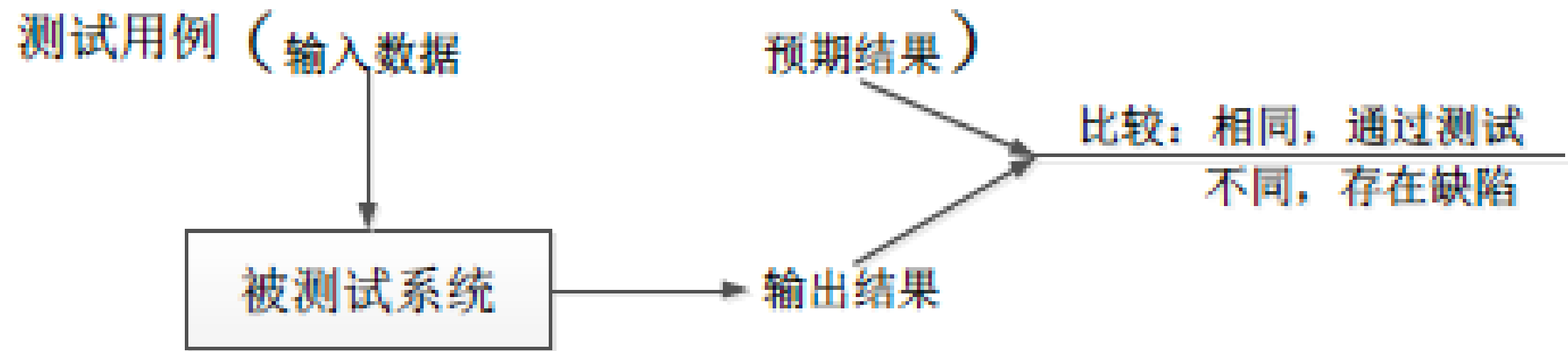
- Dynamic Techniques –
 - Testing – Based on the execution of the code on valued inputs (definition of the parameters and environmental conditions that characterize a system state must be included when necessary)
 - Profiling – A program profile records the number of times some entities of interest occur during a set of controlled executions



软件开发中的验证与确认活动

软件测试的目的

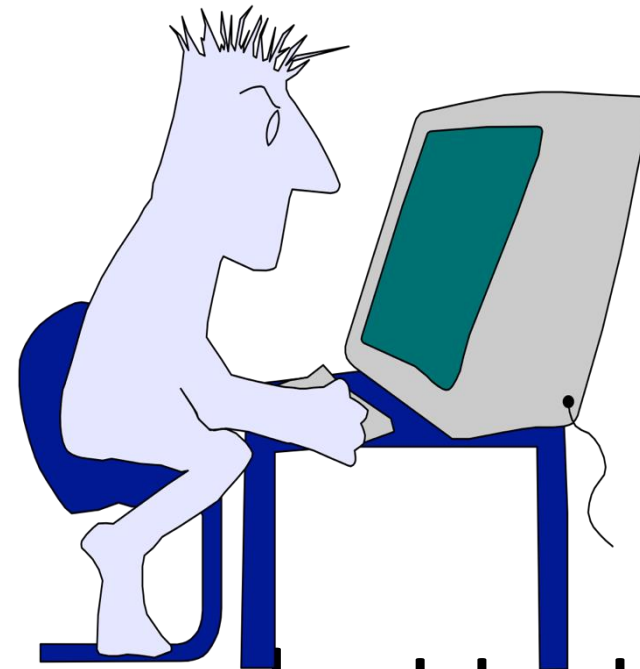
- 向开发者和用户展示软件满足了需求，表明软件产品是一个合格的产品；
 - 有效性测试
- 找出软件中的缺陷和不足。
 - 缺陷测试



测试用例



developer



independent tester

Understands the system but, will test "gently" and, is driven by "delivery"	Must learn about the system, but, will attempt to break it and, is driven by quality
---	---

大模型在软件测试中的整体作用

大模型覆盖测试全生命周期

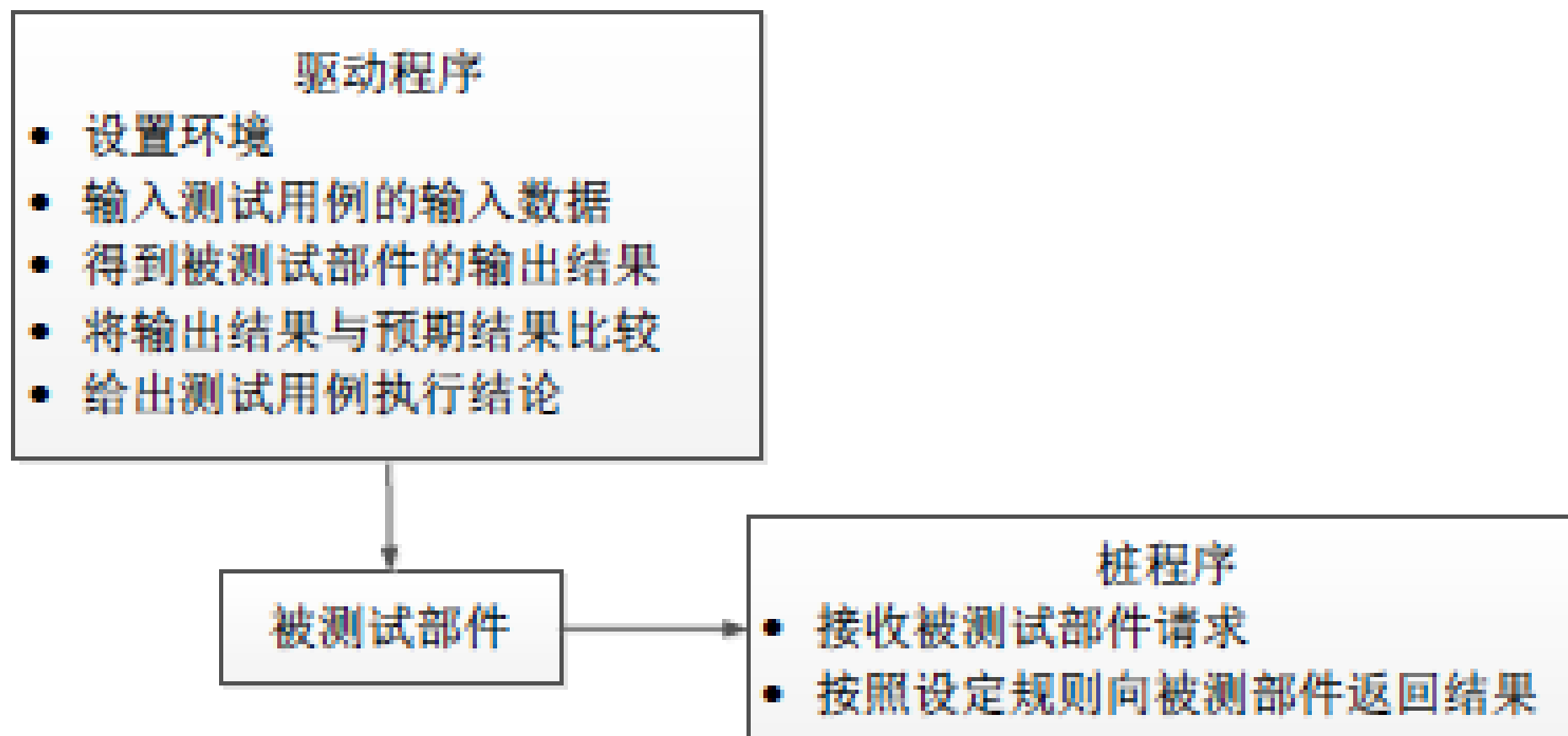
- 测试计划：根据需求自动生成测试策略、优先级和资源估算
- 测试设计：自动生成等价类、边界值、决策表、状态转换测试用例
- 测试执行：生成测试脚本（JUnit/Selenium/Pytest），支持自动化执行
- 测试评价：分析缺陷报告，预测缺陷根因，评估测试覆盖充分性

大模型作为独立测试者（Independent Tester）

- 不受开发者先验假设限制，能从用户视角发现边界和异常场景
- 无疲劳感，可持续生成大量测试用例，覆盖人工容易遗漏的场景

人机协作分工

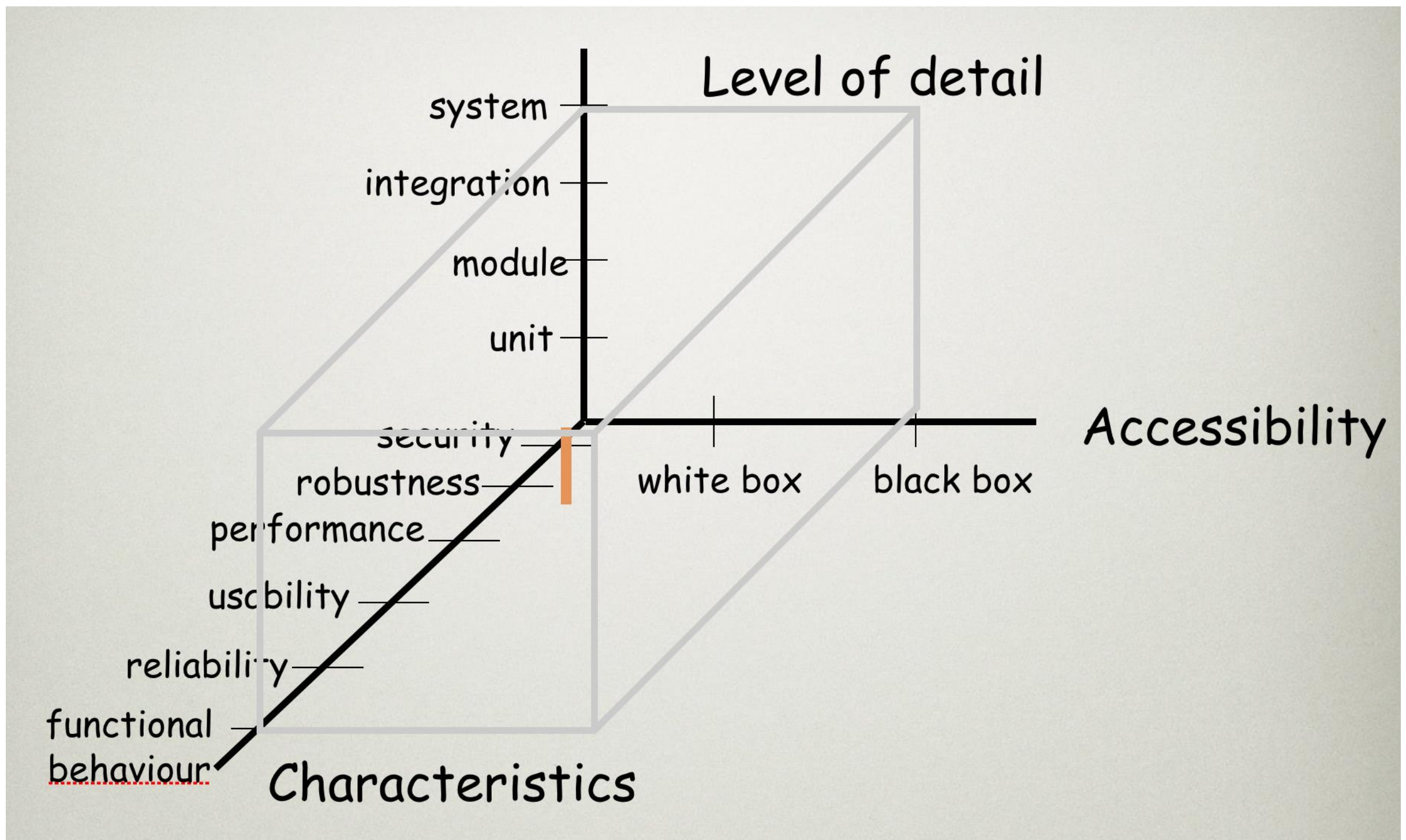
- 大模型负责：批量生成测试框架、覆盖已知等价类和边界条件
- 人工负责：业务逻辑正确性审查、探索性测试、最终测试结果判定



桩与驱动

桩程序驱动程序

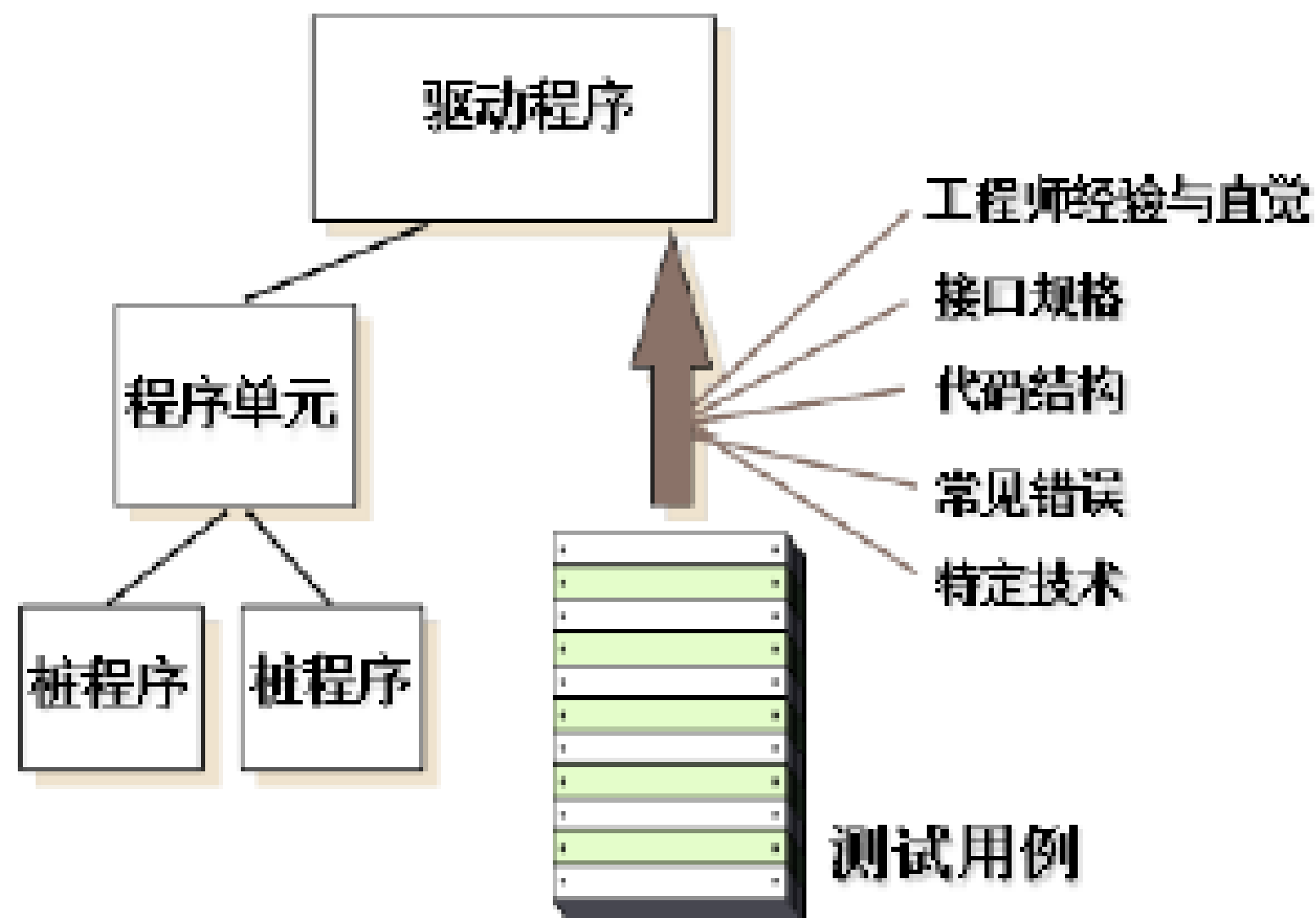
- 桩程序是被测试部件的交互环境，它扮演被测试部件需要调用的其他系统部件。桩程序对其他系统部件的扮演仅限于规格相同，内部代码要简单地多，通常是直接返回固定数据或者按照固定规则返回数据。
- 驱动程序负责创建被测试部件的执行环境，并驱动和监控被测试部件执行测试用例的过程，判定测试用例的执行结果。



TESTING Space

表 19-1 测试层次

	类别	描述
测试对象	单元测试	验证独立软件片段的功能，软件片段可以是单个的子程序、或者是由紧密联系的单元组成的较大的组件。
	集成测试	验证软件组件之间的交互。
	系统测试	关注整个系统的行为，评价系统功能需求和非功能性需求，也评价系统与外界环境（例如其它应用、硬件设备等）的交互。
测试目标	功能测试	确认观察到的被测试软件的行为是否遵从软件需求规格说明。
	验收测试	按照客户的需求检查系统。这个测试活动可能需要开发人员的参与，也可能不需要他们的参与。
	安装测试	在目标环境中通过安装来验证软件。
	α 与 β 测试	在软件发布前，让小规模、有代表性的潜在用户试用，可以在开发机构中进行（ α 测试），也可在用户处进行（ β 测试）。通常， α 与 β 测试不需要控制。
	性能测试	特别针对性能需求验证软件。
	易用性测试	评价终端用户学习和使用软件（包括用户文档）的难易程度、软件功能在支持用户任务的有效程度、从用户的错误中恢复的能力。
	可靠性测试	验证和评价系统可靠性的测试。
	安全测试	验证系统内的安全机制保护系统不受非法入侵的能力。
	恢复测试	验证软件在“灾难”后的重新启动能力。
	压力测试	以设计的最大负载运行软件，并以超过最大负载运行软件，验证软件的负载能力。
	配置测试	分析软件在规格说明的不同配置下的行为。
	回归测试	在变更系统后进行，重新执行已经测试过的测试用例集，以确保变更没有造成未预期的副作用。每个测试级别的测试（不论何种测试对象、功能还是非功能需求）都可以进行回归测试。
	其他暂未列举的测试类型.....	



单元测试

集成方向

① R,A

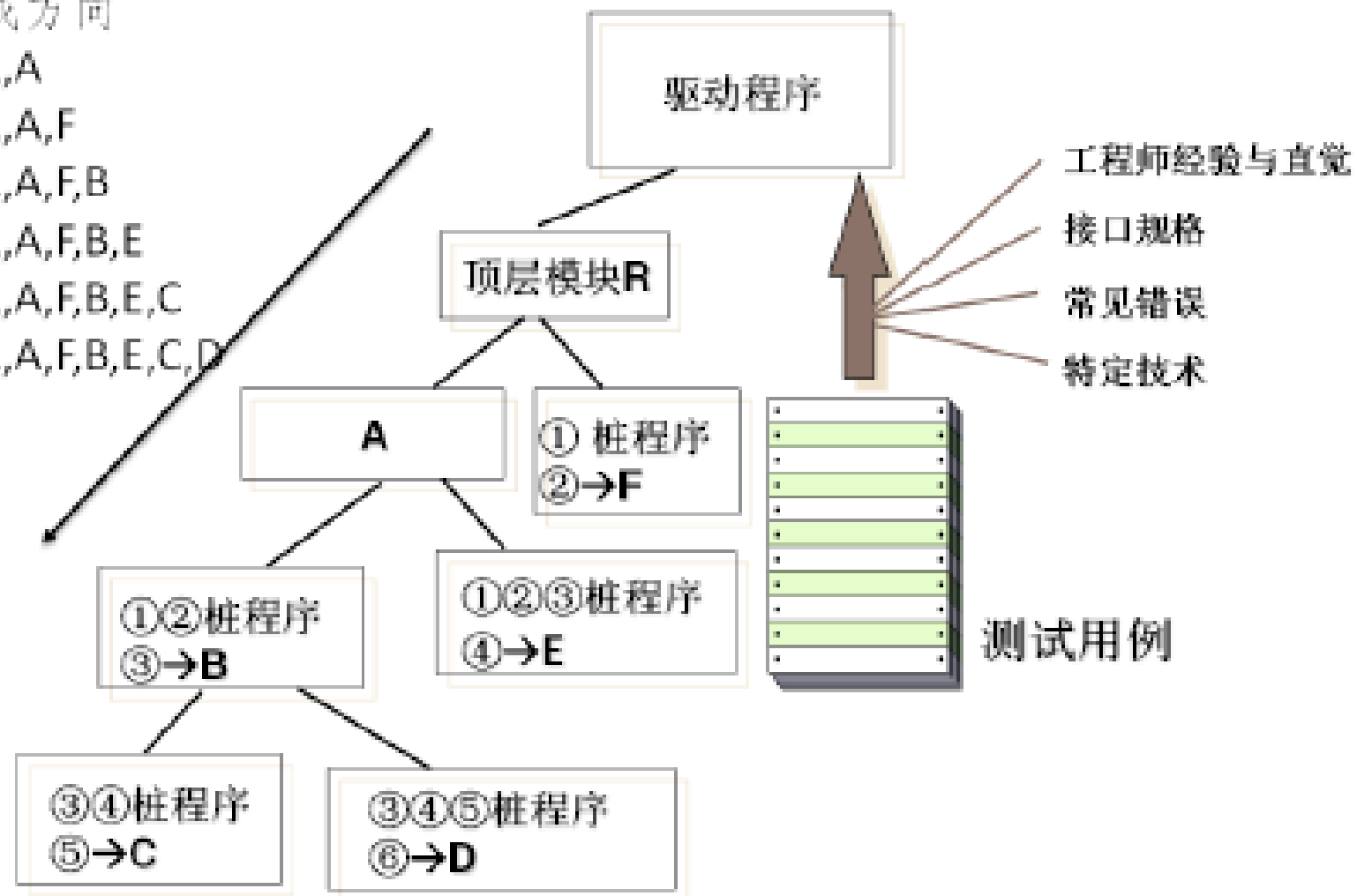
② R,A,F

③ R,A,F,B

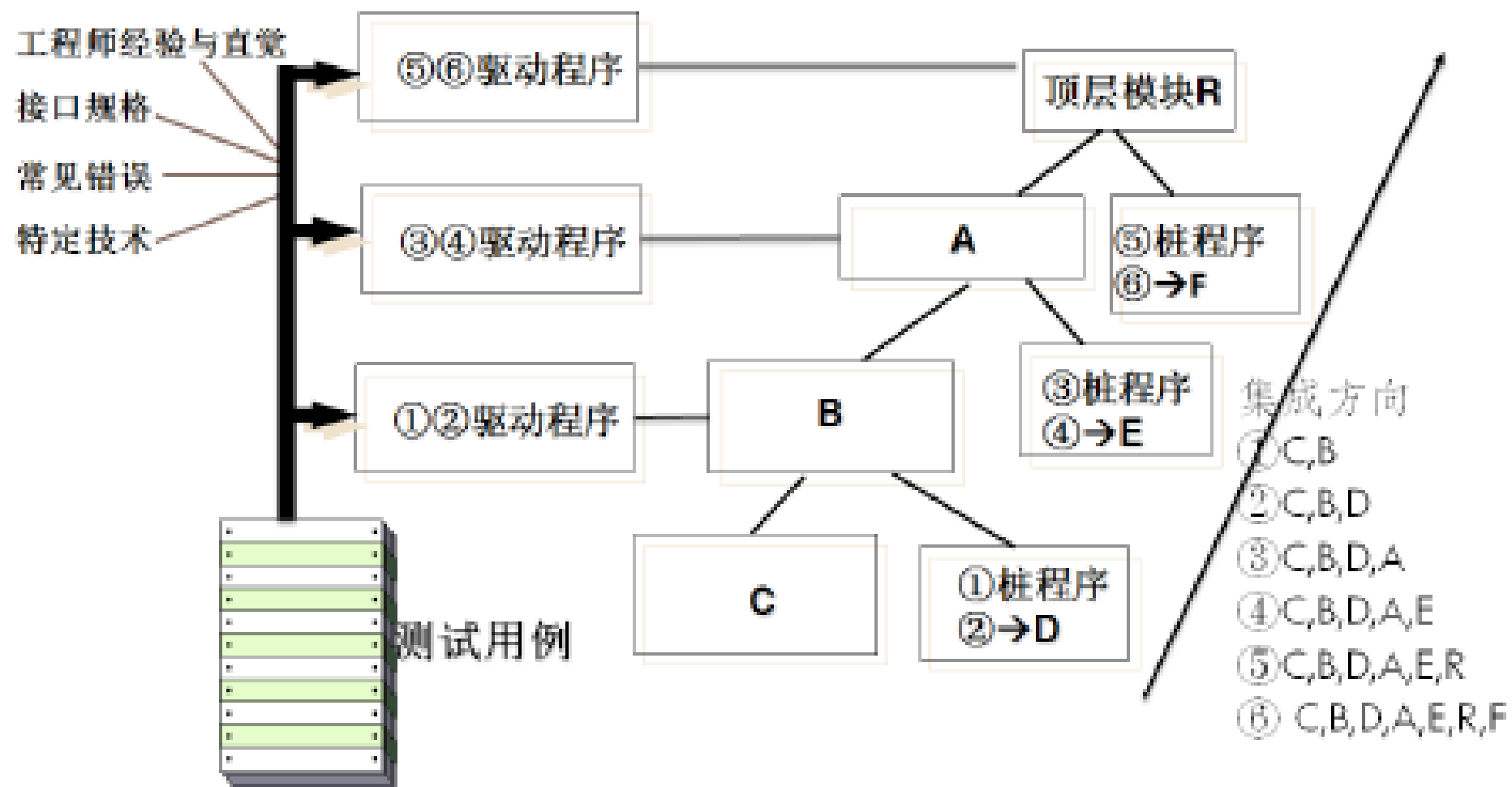
④ R,A,F,B,E

⑤ R,A,F,B,E,C

⑥ R,A,F,B,E,C,D



自顶向下的集成测试



自底向上的集成测试

系统测试

- 单元测试、集成测试更加关注技术上的正确性，重在发现设计缺陷和代码缺陷。系统测试则不同，它更关注不符合需求的缺陷和需求自身的内在缺陷。
- 根据测试目标的不同，有很多不同类型的系统测试：功能测试、非功能性测试、验收测试、安装测试等等。但是发生在软件测试阶段，完全由软件测试人员控制和执行的主要是功能测试和非功能性测试。
- 系统测试关注整个系统的行为，所以不依赖于桩程序和驱动程序。但是，使用一些测试工具可以让系统测试过程更加自动化。
- 系统测试的功能测试计划以需求规格说明文档或用例文档为基础，主要使用随机测试和基于规格的测试技术设计功能测试用例。在测试非功能性需求时需要使用针对非功能需求的特定测试技术进行测试计划和测试用例设计。

测试技术

- 测试用例的选择
- 随机测试
- 基于规格的技术 - 黑盒测试方法
- 基于代码的技术 - 白盒测试方法
- 特定测试技术

测试用例的选择

- 测试的目标是发现尽可能多缺陷， **并不**绝对要求所有。
- 因为测试是有代价的，不仅要耗费桩程序、驱动人力等成本， **更重的是**随着测试用例数量增多会直线上升。
- 所以，软件测试人员需要仔细地选择用例在代价尽可能小的情况下发现足够多的缺陷 [Zhu1997]。
- 测试技术就是帮助软件人员设计和选择用例的。

大模型辅助测试用例选择

大模型辅助智能测试用例选择

- 传统方法依赖工程师经验；大模型可系统分析规格，自动识别测试维度
- 给定需求/代码，大模型可评估现有用例的覆盖充分性并建议补充

大模型辅助测试优先级排序

- 分析历史缺陷数据，识别高风险模块，优先设计高价值测试用例
- 结合代码变更（git diff），聚焦回归测试范围，降低测试代价

大模型辅助变异测试（Mutation Testing）

- 自动生成代码变异体（改变运算符、边界条件等），检验测试用例的检错能力
- 识别'弱'测试用例——那些无法检测到变异体的测试用例

Prompt 示例

- "给定以下方法规格说明，请识别所有测试维度，生成覆盖率最高的最小测试用例集。"

随机测试

- 随机测试（ 随机测试Ad hoc Testing） 是一种 基于软件工程师直觉和经验的技术， 也许是实践中使用最为广泛的测试技术 [SWEBOK2004] 。
- 随机测试 根据软件工程师的技能、直觉和对类似程序的经验[Myers1979]， 从所有可能的输入值中选择输入子集， 建立测试用例。

表 19-2 函数 `int add(int x, int y)` 的测试用例

<i>ID</i>	<i>输入</i>		<i>预期输出</i>
<i>1</i>	<i>x=100</i>	<i>y=20</i>	<i>120</i>
<i>2</i>	<i>x=2147483640</i>	<i>y=100</i>	<i>超越最大值异常</i>

黑盒测试方法

- 等价类划分

有效等价类：是指对于程序的规格说明来合理、意义输入数据构成集合。利用有效等价类可检验程序是否实现了规格说明中所定的功能和性能。

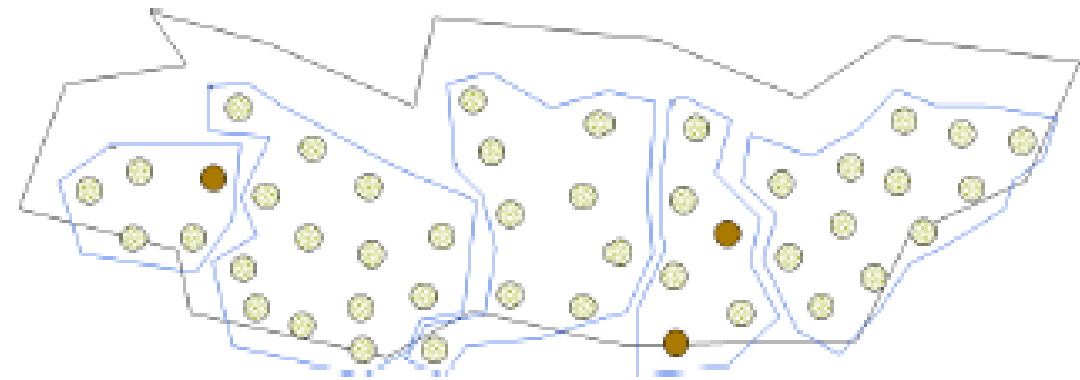


图 19-8 等价类划分

Sale类

getChange(double payment)方法

所示的规格说明，可以将其输入数据划分为三类：

- 1有效数据；
- 2无效数据， $\text{payment} \leq 0$
- 3无效数据， $\text{payment} < \text{total}$

表 19-3 Sales 类 getChange(double payment) 方法的等价类划分测试用例

ID	输入		预期输出
1	payment=100	Total=50	50
2	payment=-100	total=20	输入数据无效
3	payment=50	total=100	输入数据无效

大模型辅助等价类划分与边界值分析

大模型辅助等价类划分

- 输入方法规格说明，大模型自动识别有效/无效等价类及划分依据
- 检查遗漏的等价类：大模型能发现人工容易忽略的隐含约束

大模型辅助边界值分析

- 识别每个等价类的边界值（最小值、最小值+1、最大值-1、最大值）
- 处理复合边界：多个参数同时取边界值的组合场景

示例：getChange(double payment) 方法

- Prompt：'请为 getChange() 方法划分等价类，生成包含正常值、边界值和无效输入的完整测试用例表格'
- 大模型输出：等价类表 + 测试用例 + JUnit 代码

提升效率的实践

- 将大模型生成的等价类作为起点，人工补充领域特定的业务约束
- 大模型可直接输出 JUnit 5 参数化测试（@ParameterizedTest + @CsvSource）

黑盒测试方法

- 边界值分析

表 19-4 Sales 类 `getChange(double payment)` 方法的边界值分析测试用例

<i>ID</i>	<i>输入</i>		<i>预期输出</i>
<i>1</i>	<i>payment=50</i>	<i>total=50</i>	<i>0</i>
<i>2</i>	<i>payment=50</i>	<i>total=49</i>	<i>1</i>
<i>3</i>	<i>payment=50</i>	<i>total=51</i>	输入数据无效
<i>4</i>	<i>payment=1</i>	<i>total=1</i>	<i>0</i>
<i>5</i>	<i>payment=0</i>	<i>total=0</i>	<i>0</i>
<i>6</i>	<i>payment=-1</i>	<i>total=10</i>	输入数据无效

- ①prePoint<1000&&postPoint>=1000 → GiftLevel=1;
- ②prePoint<2000&&postPoint>=2000 → GiftLevel=2;
- ③prePoint<5000&&postPoint>=5000 → GiftLevel=3。

表 19-5 礼品赠送事件的测试用例

<i>ID</i>	<i>输入</i>		<i>预期输出</i>
<i>1</i>	<i>prePoint=500</i>	<i>postPoint=1500</i>	<i>GiftLevel=1</i>
<i>2</i>	<i>prePoint=500</i>	<i>postPoint=2500</i>	<i>GiftLevel=2</i>
<i>3</i>	<i>prePoint=500</i>	<i>postPoint=5500</i>	<i>GiftLevel=3</i>

黑盒测试方法 - 决策表

黑盒测试方法 - 状态转换

- 使用状态转换测试技术时，通常要先为对象建立状态图，描述测试对象的状态集合、输入集合和输入导致的状态转换集合。
- 以状态图基础，可建立测试对象的转换表。状态转换表每一行都应该被设计为测试用例。

大模型辅助状态转换测试

大模型辅助状态图分析与测试生成

- 输入状态图描述（文字或图），大模型自动生成状态转换表
- 基于转换表，大模型生成覆盖所有转换的最小测试用例序列
- 识别'不可达状态'和'死锁状态'等设计问题

大模型辅助 Mermaid 状态图生成

- 描述业务状态机（如订单状态：待支付→已支付→发货→完成/退款）
- 大模型输出 Mermaid 状态图代码 + 对应的测试用例序列

测试覆盖策略

- 0-Switch 覆盖：每个转换至少执行一次（大模型自动生成）
- 1-Switch 覆盖：每对连续转换至少执行一次（大模型生成更全面的序列）

Prompt 示例

- "以下是 Sale 类的状态描述，请生成状态转换表，并设计覆盖所有状态转换的 JUnit 5 测试序列。"

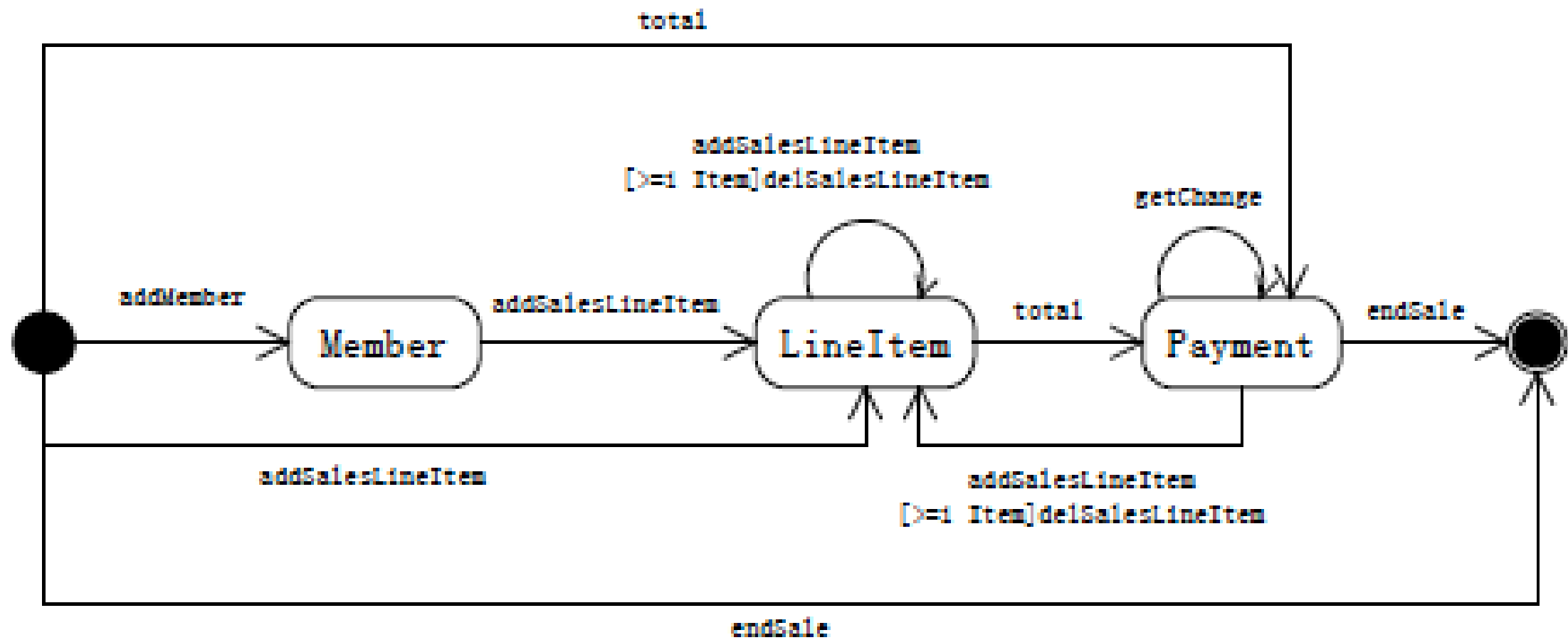


表 18-4 类 Sales 的测试用例线索

输入		预期输出状态
方法	当前状态	
<i>addMember</i>	<i>Start</i>	<i>Member</i>
	<i>Member</i>	非法
	<i>LineItem</i>	非法
	<i>Payment</i>	非法
	<i>End</i>	非法
<i>addSalesLineItem</i>	<i>Start</i>	<i>LineItem</i>
	<i>Member</i>	<i>LineItem</i>
	<i>LineItem</i>	<i>LineItem</i>
	<i>Payment</i>	<i>LineItem</i>
	<i>End</i>	非法
<i>[>=1 Item] delSalesLineItem</i>		
<i>total</i>		
<i>getChange</i>		
<i>endSale</i>		

表 18-5 类 Sales 的测试用例

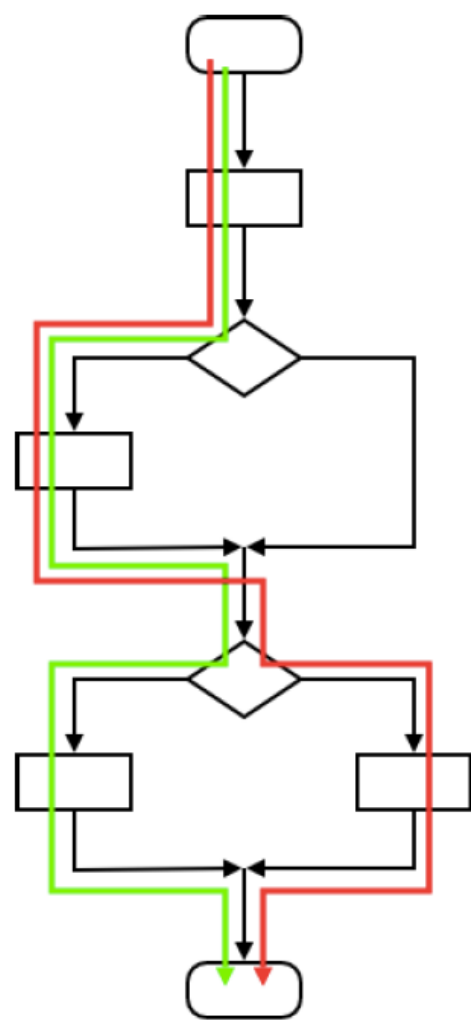
ID	輸入		預期輸出
	前置语句	方法	
1	s=new Sales();	s.addMember(2);	No Exception
2	s=new Sales(); s.addMember(1);		MemberLable Invalid Time
3	s=new Sales(); s.addSalesLineItem(1);		
4	s=new Sales(); s.addSalesLineItem(1); s.total();		
5	s=new Sales(); s.addSalesLineItem(1); s.total(); s.getChange(100); s.endSale();		Sales dose not Exists

白盒测试方法

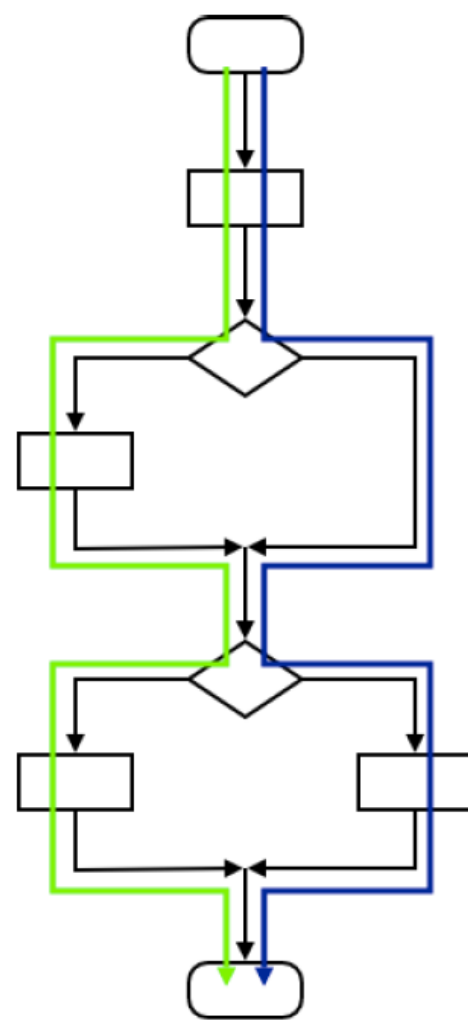
- 语句覆盖

语句覆盖设计测试用例的标准是确保被测试对象的每一行程序代码都至少执行一次。

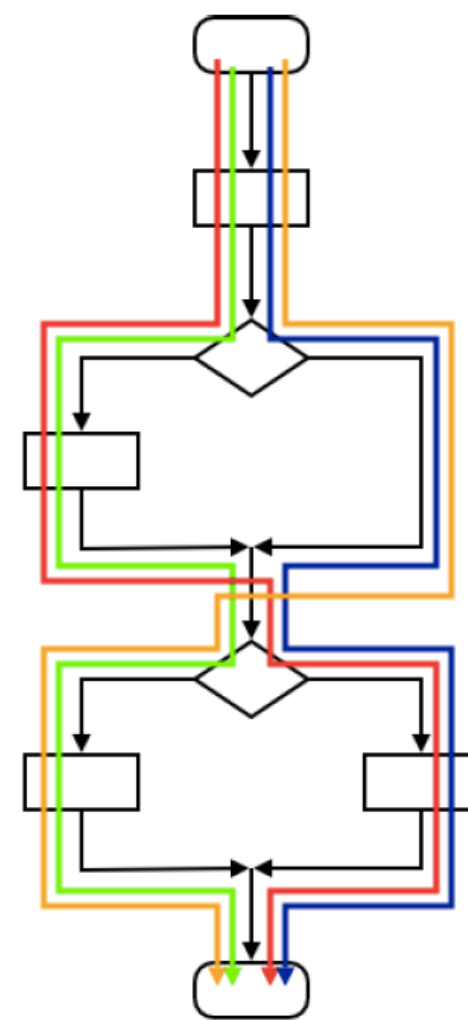
```
public Class Customer {  
    int bonus;//积分数额  
    .....  
    //预计算消费行为后的积分数额  
    int getBonus ( boolean cashPayment , int consumption , bo  
        //已有的 bonus 需要调整 getBonus, 不能直接使用  
        int preBonus=this.getBonus();  
        if (cashPayment) {  
            preBonus+=consumption;  
        }  
        if (vip) {  
            preBonus*=1.5;  
        } else {  
            preBonus*=1.2;  
        }  
        return preBonus;  
    }  
}
```



Statement Coverage
(aka line coverage)

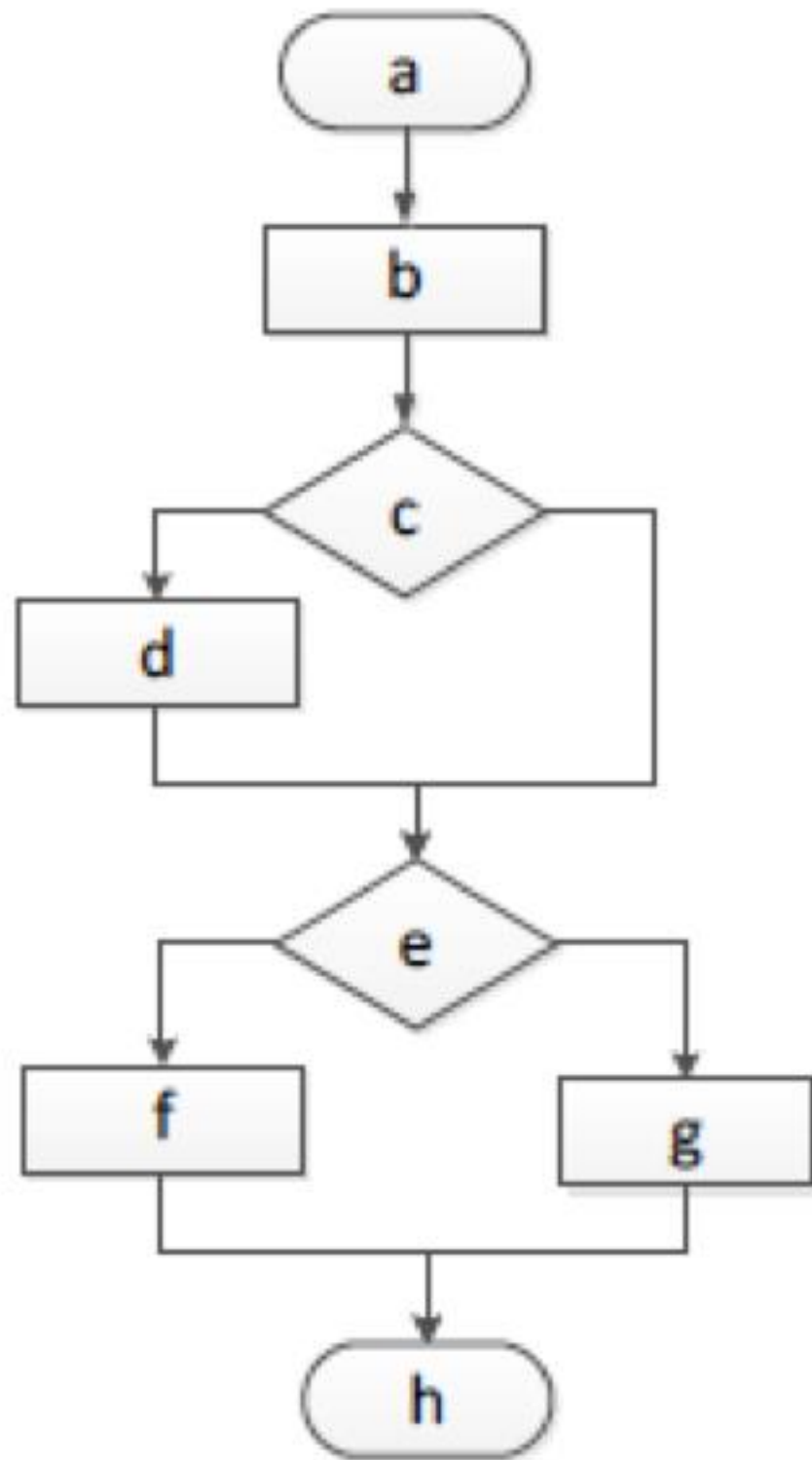


Branch Coverage
(aka condition coverage)



Path Coverage

白盒测试



为了清晰地解释对语句的覆盖，图19-10为各条语句都进行了编号。可以发现如果按照
下列两条路径执行，就能够覆盖所有的语句：
a,b,c,d,e,f,h 和
a,b,c,d,e,g,h

表 19-6 Customer.getBonus() 的语句覆盖测试用例

ID	输入				预期输出
1	<i>getBonus ()=100</i>	<i>cashPayment=true</i>	<i>consumption=100</i>	<i>vip=true</i>	<i>300</i>
2	<i>getBonus ()=100</i>	<i>cashPayment=true</i>	<i>consumption=100</i>	<i>vip=false</i>	<i>240</i>

白盒测试方法

- 条件覆盖

条件覆盖设计测试用例的标准是确保程序中每个判断的每个结果都至少满足一次

表 19-7 Customer.getBonus()的条件覆盖测试用例

ID	输入				预期输出
1	<code>getBonus ()=100</code>	<code>cashPayment=true</code>	<code>consumption=100</code>	<code>vip=true</code>	300
2	<code>getBonus ()=100</code>	<code>cashPayment=false</code>	<code>consumption=100</code>	<code>vip=false</code>	120

白盒测试方法

- 路径覆盖

路径覆盖测试用例的标准是确保程序中每条独立执行路径都至少执行一次

表 19-8 Customer.getBonus() 的路径覆盖测试用例

ID	输入				预期输出
1	<code>getBonus ()=100</code>	<code>cashPayment=true</code>	<code>consumption=100</code>	<code>vip=true</code>	300
2	<code>getBonus ()=100</code>	<code>cashPayment=false</code>	<code>consumption=100</code>	<code>vip=false</code>	120
	<code>getBonus ()=100</code>	<code>cashPayment=true</code>	<code>consumption=100</code>	<code>vip=false</code>	240
	<code>getBonus ()=100</code>	<code>cashPayment=false</code>	<code>consumption=100</code>	<code>vip=true</code>	150

大模型辅助白盒测试与覆盖率提升

大模型辅助控制流分析

- 输入代码，大模型自动识别独立路径，生成路径覆盖测试用例
- 分析圈复杂度，对高复杂度方法生成分支覆盖和条件覆盖用例

大模型辅助提升覆盖率

- 将覆盖率报告（JaCoCo）输入大模型，自动识别未覆盖分支并生成补充用例
- Prompt：'以下方法的测试覆盖率为72%，未覆盖的分支是X，请生成能覆盖这些分支的测试用例'

大模型辅助数据流测试

- 识别变量的定义-使用链（def-use pairs），生成数据流测试用例
- 检测变量未初始化使用、重复赋值未使用等潜在问题

工具集成

- JaCoCo（覆盖率） + GitHub Copilot（用例生成） + Diffblue Cover（自动化白盒测试）
- 工作流：运行测试 → 导出覆盖率报告 → 大模型分析缺口 → 生成补充测试 → 再次运行

特定测试技术

- (1) 面向对象的测试；
- (2) 图形用户接口GUI测试；
- (3) 基于Web的测试；
- (4) 基于组件的测试
- (5) 并发程序的测试；
- (6) 协议遵从性测试；
- (7) 实时系统测试；
- (8) 极端要求安全性的系统的测试。

面向对象测试技术

- 在本书中使用了一些面向对象的测试技术：
 - (1) 7.5使用用例和场景设计系统测试用例；
 - (2) 11.5基于协作设计类之间的集成测试用例；
 - (3) 18.5基于状态图设计类的单元测试用例。

测试活动

- 测试计划
- 测试设计
- 测试执行
- 测试评价

大模型辅助测试活动全流程

测试计划阶段

- 根据需求规格，大模型生成测试策略、测试范围、资源估算建议
- 识别高风险模块，建议测试优先级和测试类型分配

测试设计阶段

- 自动生成黑盒测试用例（等价类/边界值/决策表/状态转换）
- 自动生成白盒测试用例（语句/分支/路径覆盖），输出 JUnit/Pytest 代码
- 自动生成 Mock 对象和测试桩，隔离被测组件的外部依赖

测试执行阶段

- 生成 Selenium/Playwright UI 测试脚本，实现端到端自动化测试
- 分析测试失败日志，定位缺陷根因，区分测试代码错误与被测代码缺陷

测试评价阶段

- 分析覆盖率报告，识别测试盲区并建议补充测试
- 聚合缺陷数据，识别缺陷模式（如高频出现的缺陷类型或模块）

表 19-9 测试用例日志

测试用例 ID	种类	条件	期望结果	测试结果	测试对象 ID
输入测试用例 ID	输入测试种类	输入特殊的测试条件	测试用例预期的结果	记录“通过”或者“失败”	被测试对象的 ID

测试用例日志

表 19-10 缺陷报告

缺陷ID	发现日期	测试脚本	测试用例	期望结果	实际结果	状态	严重性	优先级	缺陷类型	备注

缺陷报告

测试层次的测试报告

1.引言

1.1 文档标识

文档的唯一标识，可以包含日期、单位、作者、状态、版本等信息。

1.2 范围

文档的内容及组织方式。

1.3 参考资料

2.详细情况

2.1 测试结果概述

总结对具体测试项目的评价。

标明测试项目（对象），注明它们的层级，描述测试活动执行的环境和环境带来的影响。

2.2 详细测试结果

汇总测试的结果。

- 标明所有已解决的缺陷，并总结它们的分析辨别（Resolution）。
- 标明所有未解决的缺陷。对于推迟解决的缺陷，解释后续的处理办法。
- 总结主要的测试活动和事件。总结收集的相关度量数据。
- 报告测试对象与它们规格之间的任何差异。描述每个差异的原因。

如果有相应的测试计划，就依据测试计划中的综合性标准进行测试过程的综合性评价。

2.3 决策理由

描述影响总结决定的相关事宜。

2.4 总结与建议

为每一个测试对象：

- 描述整体评价，包括它的局限性。评价以测试结果和测试项目所处层次的通过/失败标准为基础。评价还可能包括对失败风险的估计。
- 建议如果要投入生产使用应该具备的状态和所处的环境。

3.其他

3.1 术语表

3.2 文档修改历史

图 19-11 [EEE829-2008]建议的具体测试层次的测试报告

总测试报告

1.引言

1.1 文档标识

文档的唯一标识，可以包含日期、单位、作者、状态、版本等信息。

1.2 范围

文档的内容及组织方式。

1.3 参考资料

2.详细情况

2.1 总测试结果概述

总结每一个测试层次的评价信息：

- 测试活动总结：描述所有测试活动的执行汇总情况。
- 测试任务的结果总结：描述所有的测试任务情况。
- 缺陷和辨析（Resolution）总结：分类总结测试中发现的缺陷。已解决的缺陷和未解决的缺陷应该分开总结。
- 评估产品质量：对产品进行总体评价。
- 总结收集的度量数据。

2.2 决策理由

描述对软件作出“通过/不通过/有条件通过”决定的原因。

2.3 总结与建议

- 描述对产品的总体性评价。评价可能会包括对失败风险的估计。
- 建议如果产品要投入生产使用应该具备的状态和所处的环境。
- 提供关于产品验收的总结和建议。总结包括确定产品准备就绪需要的精力（effort）汇总。建议要标明判定产品已就绪的指示特征。
- 描述学习到的任何经验。这部分可能会包含过程改进的信息。
- 如果有缺陷被推迟处理，要解释后续处理过程。

3.其他

3.1 术语表

3.2 文档修改历史

图 19-12 [IEEE829-2008]建议的总测试报告

测试度量

- 缺陷数据
- 测试覆盖率
 - 需求覆盖率
 - 模块覆盖率
 - 代码覆盖率

大模型辅助测试度量与缺陷分析

大模型辅助缺陷数据分析

- 输入缺陷报告集，大模型识别缺陷分布规律（高发模块、引入阶段、缺陷类型）
- 预测新代码中的高风险区域，指导测试资源的重点分配
- Prompt：'分析以下50条缺陷报告，总结缺陷分布规律并建议优先测试的模块'

大模型辅助覆盖率分析

- 解读覆盖率数据（语句/分支/路径），判断当前覆盖水平是否满足质量目标
- 识别'高覆盖低效率'问题：覆盖率高但检错能力弱的测试用例

大模型辅助测试报告生成

- 整合测试执行结果、覆盖率数据、缺陷统计，自动生成结构化测试报告
- 针对不同受众（开发者/项目经理/客户）生成不同详细程度的报告

持续质量改进

- 对比多次迭代的缺陷数据，识别质量趋势（改善/恶化），及时预警
- 大模型分析历史测试数据，建议测试规范改进，形成持续改进闭环

缺陷度量

- 根据引入缺陷的阶段
 - 系统需求缺陷、设计缺陷和编码缺陷
- 根据缺陷的影响力
 - 严重、一般和无影响

测试智能体（Testing Agent）概述

什么是测试智能体

- 在大模型基础上增加自主规划、工具调用、多步执行能力的自动化测试系统
- 不只是单轮问答：能持续感知环境、执行动作、观察反馈、迭代优化
- 典型循环：Observe（观察代码/测试结果）→ Plan（制定测试策略）→ Act（生成/执行测试）→ Reflect（分析失败、改进）

与普通大模型提示词的本质区别

- 普通提示词：单轮输入输出，人工驱动每一步
- 测试智能体：多步自主、工具调用（代码执行/版本控制/CI）、闭环反馈
- 智能体可跨越多个文件、模块，端到端完成「发现问题→生成测试→修复→验证」

智能体在软件工程中的定位

- 人工层：需求分析、测试策略定义、最终审核 —— Human-in-the-Loop
- 智能体层：测试用例生成、执行、失败分析、修复循环、报告生成
- 工具层：测试框架、代码分析器、CI/CD、覆盖率工具等

测试智能体的核心技能 (Skills)

Skill 1：代码理解与调用图分析

- 解析函数调用链、数据流，识别模块间依赖
- 工具：Tree-sitter 语法解析 + 大模型语义理解，建立调用图做测试范围判断

Skill 2：测试用例自动生成

- 基于函数签名、文档注释、已有测试自动补充新用例
- 覆盖正常路径、边界值、异常场景，生成参数化测试 (@pytest.mark.parametrize / JUnit @ParameterizedTest)

Skill 3：测试执行与自修复 (Self-Repair)

- 调用 pytest/JUnit 等执行测试，捕获失败信息
- 分析报错 Traceback，定位根因，自动修改测试或被测代码，迭代直到通过
- 典型案例：SWE-agent 在 SWE-bench 上实现自主修复循环，无需人工介入

Skill 4：缺陷定位与根因分析

- 将失败测试映射到源码位置，生成最小复现用例
- 推断是测试写错还是被测代码存在缺陷，给出修复建议

Skill 5：覆盖率驱动补全

- 读取覆盖率报告 (lcov/coverage.py)，识别未覆盖分支
- 针对空白分支自动生成测试，迭代直到覆盖率达到预设阈值

主流测试智能体工具与框架

SWE-agent (Princeton/MIT 开源)

- 以 Agent-Computer Interface 调用 shell/编辑器，自主解决 GitHub Issues
- 在 SWE-bench 上实现对真实开源项目 Bug 的自动修复，测试生成是其关键步骤

OpenDevin / All-Hands AI (开源)

- 支持「写代码→生成测试→执行→修复」完整循环
- 在浏览器沙盒中运行，可读文档、查 API、验证测试，适合复杂项目

Diffblue Cover (商业，专注 Java)

- 深度集成 IntelliJ / Maven / CI，分析字节码生成高覆盖率、可维护的 JUnit 测试
- 支持增量更新：代码变更时自动重生对应测试，降低测试腐败成本

GitHub Copilot Workspace

- 以自然语言描述任务，Copilot 自动规划、生成代码与测试、提交 PR
- 将「Issue → 测试 → 实现 → Review」整合为一个智能体工作流

AutoGen / LangChain 测试流水线 (开源框架)

- 多智能体协同：规划 Agent + 生成 Agent + 执行 Agent + 报告 Agent
- 可定制流水线，连接各种测试工具和 CI 系统，适合企业自研测试智能体

Claude Code 作为测试智能体——实战示例

场景 1：为现有模块一键生成单元测试

- `$ claude "为 src/payment/checkout.py 生成完整的 pytest 单元测试，覆盖正常路径、边界值和异常场景"`
- Claude Code 自动读取源文件，理解业务逻辑，生成带 mock 的测试文件并写入 `tests/test_checkout.py`

场景 2：CI 失败后自主修复

- `$ claude "CI 中 test_checkout.py::test_refund_negative 失败，错误信息如下：[粘贴 Traceback]，请修复"`
- Claude Code 分析 Traceback，定位 checkout.py 第 87 行逻辑缺陷，生成修复 patch，重新运行测试验证

场景 3：覆盖率提升任务

- `$ claude "运行 pytest --cov=src/payment，当前覆盖率 61%，补充测试使分支覆盖率达到 80%"`
- Claude Code 读取 coverage.xml，识别 12 个未覆盖分支，逐一生成针对性测试并追加到测试文件

场景 4：多轮自主迭代（`--dangerously-skip-permissions`）

- `$ claude --dangerously-skip-permissions "对整个 src/ 目录生成测试，执行，修复所有失败，直到全部通过"`
- 智能体自主循环：生成→执行→分析失败→修复→重测，无需人工介入，适合 CI 夜间自动运行

Claude Code 测试 Skills 配置实例

CLAUDE.md : 为项目定制测试智能体行为

- 在项目根目录创建 CLAUDE.md, 声明测试规范, Claude Code 每次启动自动加载
- # 测试规范
- - 测试框架: pytest + pytest-cov; Mock 使用 unittest.mock
- - 覆盖率要求: 语句覆盖 $\geq 80\%$, 分支覆盖 $\geq 70\%$
- - 禁止在测试中使用真实数据库, 统一使用 fixtures/conftest.py 中的内存 DB
- - 生成测试后必须运行 `pytest -x` 验证, 失败则自动修复再提交

自定义 Slash 命令: /test-gen

- 在 `.claude/commands/` 目录创建 `test-gen.md`, 定义专属测试生成命令
- # /test-gen
- 读取 `$ARGUMENTS` 指定的 Python 文件, 分析所有 public 函数,
- 为每个函数生成 3 类测试: 正常用例、边界值用例、异常用例,
- 输出到 `tests/` 目录, 命名规则 `test_[原文件名].py`, 最后运行 `pytest` 验证。
- 使用方式: 在项目内执行 `/test-gen src/user_service.py`

Hooks : 测试自动触发 (settings.json)

- "hooks": { "PostToolUse": [{ "matcher": "Write",
- "hooks": [{ "type": "command",
- "command": "pytest tests/ -x -q 2>&1 | tail -5" }] }
- 每次 Claude Code 写入文件后自动运行测试, 输出最后 5 行结果, 失败则进入修复循环

测试智能体端到端工作流示例

场景：CI 流水线中一个失败的集成测试

- 触发：PR 提交后 CI 检测到 1 个 Test Failure，智能体自动介入

Step 1 感知 (Observe)

- 读取失败日志、Traceback，定位涉及文件与函数
- 拉取相关代码上下文（调用链 ± 3 层），建立问题全貌

Step 2 规划 (Plan)

- 判断：是测试断言错误、接口变更，还是被测逻辑 Bug？
- 制定修复策略：优先修复被测代码，同步更新测试

Step 3 行动 (Act)

- 生成修复 Patch 或新测试用例，调用 pytest 执行验证
- 若仍失败：分析新的 Traceback，进入下一轮迭代（平均 2-4 轮收敛）

Step 4 反思与提交 (Reflect & Commit)

- 所有测试通过后，生成 Commit Message，提交 Draft PR
- 附带：变更说明、受影响测试列表、覆盖率变化报告

Human-in-the-Loop 检查点

- 人工 Review：PR 内容、测试充分性、逻辑正确性 —— 最终合并决策由人完成

测试智能体的局限与最佳实践

主要局限

- 幻觉风险：生成的测试看似合理但断言错误，造成覆盖率高但扫错能力弱（Test Oracle Problem）
- 上下文窗口限制：大型代码库中跨文件推理能力下降，易遗漏全局约束
- 环境依赖：测试执行需真实沙盒，配置复杂（数据库、外部服务 Mock）
- 可维护性：生成的测试风格不统一，长期维护成本需评估

Human-in-the-Loop (HITL) 最佳实践

- 智能体负责「生成+初步验证」，人工负责「策略定义+最终审核」
- 关键模块（安全/支付）保留人工测试用例，不完全依赖智能体
- 建立测试质量门禁：覆盖率、变异测试分数（Mutation Score）作为 HITL 触发阈值

与 CI/CD 融合的推荐架构

- PR 提交 → 智能体自动补全测试 → CI 执行 → 覆盖率不足则智能体继续迭代
- 将智能体生成的测试纳入版本控制，与手工测试统一管理

评估测试智能体质量的维度

- 缺陷检出率（Mutation Score）、覆盖率增量、误报率、用例可读性

表 19-11 测试覆盖率与程序规模，源自[Jones2007]

规模（代码行）	测试用例数量	测试覆盖率
1	1	100%
10	2	100%
100	5	95%
1000	15	75%
10000	250	50%
100000	4000	35%
1000000	50000	25%
10000000	350000	15%

测试覆盖率与程序规模

覆盖率

- (1) 需求覆盖率 = 被测试的需求数量 / 需求总数 ;
- (2) 模块覆盖率 = 被测试的模块数量 / 模块总数 ;
- (3) 代码覆盖率 = 被测试的代码行 / 代码行数总数 。